stateless기반 누적치 구현관련 메모

> 해당 내용 재무 에이전트 내부에서 변수 전달용도로 구현했지만, 이 '변수로 저장'해두는 방식은 추후 **지역코드를 통한 지도-에이전트 연동** 용도로도 사용할 수 있어보인다.



누적치 반영 issue가 있어 오늘에야 개선... 임대료 200만 입력했는데 이전 입력인 매출 700을 포함한 데이터로 시뮬레이션하는 모습이다.(초기-기본값은 1,400)

또한 해당 내용은 변수들을 localstorage에 저장하는 방식으로 구현되어있다.
추후 회원제 도입 같은 상황에 cosmosDB등으로 변경 될 수 있다는 점은 고려할 만 하다.
다만 해당 경우에도 Back-End의 변경점은 크게 없을 것으로 파악중.

본론

전체적인 입/출력에 현재 상태(current_*params*) *및 상태 업데이트*(*update_params*)가 추가된다는 점을 확인하자
서술되는 상태는 재무 한정으로 작동하게 되어있어(if domain == "finance" 같은 부분들...), 전체 에이전트 연동 및 각 에이전트에는 적절히 변경하자

api_server

```
class QueryRequest(BaseModel):
    question: str
    session_id: str | None = Field(default=None)
    founder_context: str | None = Field(default=None)
    domain: str | None = Field(default=None)
    max_retries: int = Field(default=3, ge=0, le=10)
```

```
    current_params: dict | None = Field(default=None)  # ← 추가
```

입력에 현재상태(기본값=None) 추가

```
result = await orchestrator.run(
    domain=domain,
    question=req.question,
    profile=session["profile"],
    session_id=sid,
    max_retries=req.max_retries,
    session_vars=session["extracted"] if session["extracted"] else None,
    current_params=req.current_params,  # ← 추가
)
```

query 함수 내부의 orchestrator.run() 호출시에도 변수를 추가해준다
물론 run()함수에도 현재 상태를 받도록 추가해야한다.

```
        return {
            ... 기존 return 변수들...
            "chart":             result.get("chart"),  # ← 차트 추가
            "updated_params": result.get("updated_params"), # ← 변수 추가
            ... 기존 return 변수들...
            ),
```

갱신치도 당연하게 return에 추가

##

orchestrator.run()

```
async def run(
    domain: Literal["admin", "finance", "legal", "location"],
    question: str,
    profile: str = "",
    session_id: str = "",
    max_retries: int = 3,
    session_vars: dict | None = None,
    current_params: dict | None = None,  # ← 추가
) -> dict:
```

입력추가

```
request_id = str(uuid.uuid4())[:8]
rejection_history = []
retry_prompt = ""
draft = ""
prev_draft = None
chart = None # ← chart 추가
updated_params = None # ← 추가
```

최초 변수 초기화 부분.

(chart삽입을 동시에 확인중이라 chart도 포함되어있다. 해당내용은 답변에 그래프(등의 추가 데이터) 추가용 메모 참조)

```
# extra = {"session_vars": session_vars} if domain == "finance" and session_vars else {}
# 위 내용 변경
extra = {}
if domain == "finance":
    if session_vars:
        extra["session_vars"] = session_vars
    if current_params:
        extra["current_params"] = current_params  # ← 추가
```

extra 내용 변경. session_vars 같은 경우 기존 사용하던 누적치 관련 부분이라... 둘 중 하나는 제거하지 않을까 싶다.

```
        # finance는 dict, 나머지는 문자열 반환
        if domain == "finance":
            draft = rawdraft["draft"]
            chart = rawdraft.get("chart")
            updated_params = rawdraft.get("updated_params")
        else:
            draft = rawdraft
            chart = None
            updated_params = None  # ← 여기서도 선언
```

chart삽입때도 했었던 draft+chart에 updated_params 까지 넘긴다.

```
        if verdict["approved"]:
            return {
            ... 기존 return 변수들...
                "draft":            draft,
                "chart": chart if domain == "finance" else None,
                "updated_params": updated_params,  # ← 추가
            }
```

```
return {
      ... 기존 return 변수들...
```

```
        "updated_params": updated_params,
    }
```

각종 return파트에 마찬가지로 update한 변수들 추가.

##

agent

변수 추출에 있어서는 기존부터 재무 에이전트가 사용하던 방식이 유지된다.
해당 내용은 재무 에이전트 및 플러그인 파일을 참고(_PARAM_EXTRACT_PROMPT , _extract_params 등...)

##

front에서의 활용



현재는 상태확인용 패널 구현이긴 하지만, 변수 자체를 front에서 사용할 수 있다(전달했으니까...)

JS

```
const DEFAULT_PARAMS = {
    "revenue": [14000000],
    "cost": null,
    "salary": null,
    "hours": null,
```

```
    "rent": null,
    "admin": null,
    "fee": null,
    "initial_investment": null
};
# default 지정

let currentParams = loadParams();
let isLoading = false;
let panelVisible = true;

// localStorage 저장/불러오기 >> 해당 부분 다른(DB등)방식으로 변경될 수 있음.
function loadParams() {
  try {
    const saved = localStorage.getItem('financeParams');
    return saved ? JSON.parse(saved) : { ...DEFAULT_PARAMS };
  } catch { return { ...DEFAULT_PARAMS }; }
}
function saveParams(params) {
  localStorage.setItem('financeParams', JSON.stringify(params));
}

# 패널로 보이는 부분, 패널 구현이 목적이 아니라면 pass.
function renderParams() {
  const list = document.getElementById('paramList');
  const fmt = (v) => {
    if (v === null || v === undefined) return { text: 'null', cls: 'null-val' };
    if (Array.isArray(v)) {
      const avg = Math.round(v.reduce((a,b)=>a+b,0)/v.length/10000);
      return { text: `[${v.length}개] avg ${avg.toLocaleString()}만`, cls: '' };
    }
    return { text: Math.round(v/10000).toLocaleString() + '만원', cls: '' };
  };

  const sections = {
    '매출': ['revenue'],
    '비용': ['cost', 'salary', 'hours', 'rent', 'admin', 'fee'],
    '기타': ['initial_investment'],
  };

  list.innerHTML = Object.entries(sections).map(([sec, keys]) => `
    <div class="panel-section">${sec}</div>
    ${keys.map(k => {
      const { text, cls } = fmt(currentParams[k]);
      return `<div class="param-item">
        <span class="param-key">${k}</span>
        <span class="param-val ${cls}" title="${JSON.stringify(currentParams[k])}">${text}</span>
      </div>`;
    }).join('')}
  `).join('');
}
```

```
# 파라미터 업데이트(서버가 updated_params 반환)
if (data.updated_params) {
    currentParams = { ...currentParams, ...data.updated_params };
    saveParams(currentParams);
    renderParams();  // ← 패널 갱신
}
```

지역코드를 통한 지도-에이전트 연동 용도용으로는, 중간 updated_params 부분을 패널이 아닌 지도 위치 조정으로 사용할 수 있지 않을까 싶다.

저장된 지역코드>좌표이동 등